Elevator Vertical Distance Estimation

Eyal Yakir Oria

April 16, 2026

Abstract

1 Introduction

2 Data Collection

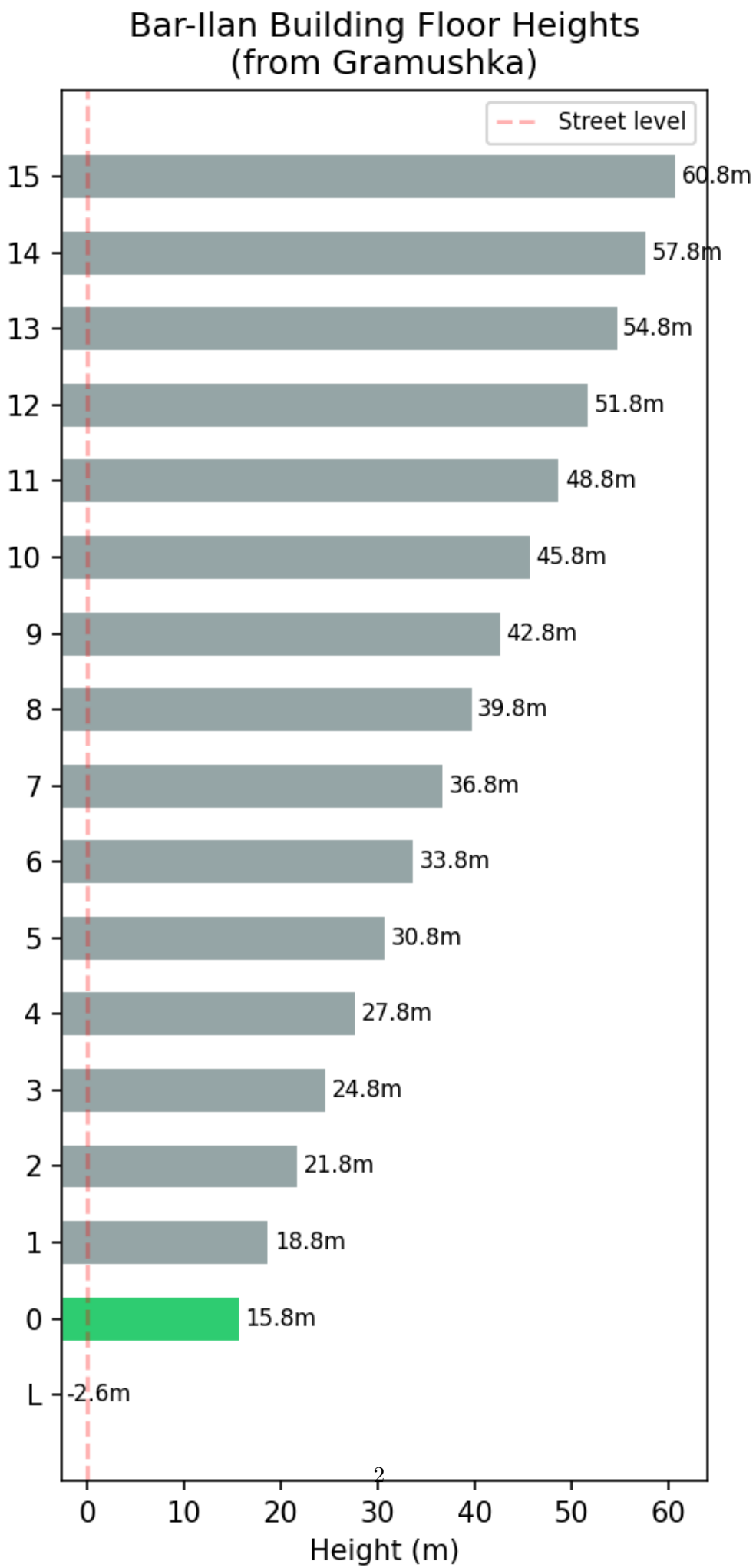
This section documents, end to end, how the dataset underlying this work was gathered, how it is turned into a usable ground truth, and the engineering problems we had to solve along the way. It is written so that a developer who has never touched the repository can reproduce the pipeline from scratch by reading this section alongside `src/data/README.md`.

2.1 Experimental Design

We ran a controlled field campaign across **eight different buildings** in central Israel, each with a different elevator, cabin geometry, motor characteristic, and floor-to-floor height. The goal of the campaign was to obtain many statistically-independent elevator rides covering a realistic distribution of buildings (low-rise residential, mid-rise office, hotel, university) so that any algorithm trained or evaluated on this data is not just fitted to a single elevator. The buildings sampled in this campaign are listed in Table 1; Figure 1 summarises their height distribution.

Building (internal name)	Type	Location
`milleniumHotel`	Hotel	Tel Aviv
`acroBuilding`	Office	Tel Aviv
`beitMansour1`	Residential	Ra'anana
`beitYitzchakiRaanana`	Residential	Ra'anana
`BarIlan2Herzelia`	Residential	Herzeliya
`Haari3`	Residential	**[TODO: location]**
`[TODO: building_7]`	**[TODO: type]**	**[TODO: location]**
`[TODO: building_8]`	**[TODO: type]**	**[TODO: location]**

Table 1: The eight buildings used in the field campaign. Internal names match the `location` field in `rawData/` experiment folders.


Bar-Ilan Building Floor Heights
(from Gramushka)
Street level
15
14
13
12
11
10
9
8
7
6
5
4
3
2
1
0
L
60.8m
57.8m
54.8m
51.8m
48.8m
45.8m
42.8m
39.8m
36.8m
33.8m
30.8m
27.8m
24.8m
21.8m
18.8m
15.8m
-2.6m
0
10
20
30
40
50
60
Height (m)

Phones. In every ride we simultaneously recorded data from **four phones**. Using four devices per ride serves two purposes. First, it gives us heterogeneous sensor hardware on a single, perfectly synchronised physical event, which lets us study cross-device variance (different MEMS accelerometers, different sampling jitter, different barometer quality) without confounding it with different rides. Second, it guarantees that at least one of the four devices carries a **barometric pressure sensor** that we can use as a ground-truth source, even when one of the target phones under evaluation does not have one. Typical device mix across the campaign:

- `SamsungSM-S911B` — Galaxy S23 (has barometer).
- `GooglePixel10` / `GooglePixel` — Pixel family (has barometer).
- `Xiaomi22101320I` — Redmi Note 12 family (no barometer).
- `SamsungSM-A235F` — Galaxy A23 (no barometer).
- `GalaxyZFlip6` — foldable (has barometer).

The experimenters (`eyalyakir`, `UriyaCohenEliya`, `RoyTurgeman`) carried the devices together in a single hand or bag so that all four phones experience nearly identical motion, vibration, and acoustic environment during a ride.

Folder and file naming. Every recording is stored under `src/data/rawData/<experimenter>_<location>` to make the origin of every sample unambiguous from the path alone. This convention is documented in `src/data/README.md` and is enforced only by convention — the loader (§2.5) does not parse it, but every analysis notebook groups by it.

2.2 Logging: The Barometer

On every phone, a sensor-logging Android application subscribes to every inertial and environmental sensor the device exposes and dumps every sample, verbatim, to a tab-separated text file `sensorLog_<ISO-timestamp>.txt`. One row per sample, no header, format:

$$\texttt{<timestamp\_ms>\textbackslash t<SENSOR>\textbackslash t<v}_1\texttt{>\textbackslash t<v}_2\texttt{>...}$$

The sensors we rely on are summarised in Table 2.

The most important channel for ground truth is the **barometer (`PRS`)**. The pressure sensor is sampled in parallel with the other sensors and is the sole signal the elevator detector (§2.3) acts on. Because not every device in the fleet has a barometer, we attach a secondary device (typically a Pixel or S23) that *does* have one to every ride whose primary phone is pressure-less; its recording lives in a `forBarometer/` sub-folder under the primary experiment's directory. This secondary-device trick is central to the time-alignment problem discussed in §2.4.

Tag	Sensor	Columns
`ACC`	Linear accelerometer (SI-compensated)	`x, y, z`
`GYR`	Gyroscope	`x, y, z`
`MAG`	Magnetometer	`x, y, z`
`RAWGYR`	Uncalibrated gyroscope	`x, y, z, bias_x, bias_y, bias_z`
`RAWMAG`	Uncalibrated magnetometer	`x, y, z, bias_x, bias_y, bias_z`
`ORI`	Rotation-vector quaternion	`w, x, y, z`
`PRS`	Barometer	`pressure` (hPa)
`GPS`	GPS fix	`lat, lon, alt`

Table 2: Per-sensor column schema emitted by the logger. Any row whose column count does not match its sensor tag is dropped silently by the parser (`src/data/loader/parsing.py`).

Once the recording is over, the raw dump is committed to `rawData/<exp_name>/`, together with a hand-filled `metadata.txt` (experimenter, phone, location, description, date, time). The loader pipeline (§2.5) then materialises a CSV per sensor under `structuredData/data/<exp_name>/` and derives `gt.csv` — the ground truth used by every downstream algorithm.

2.3 Elevator Detection

Ground truth for this project is a labelling of the timeline into `up`, `down`, and `outside` intervals: "outside" means the phone is not on a moving elevator (waiting, walking, idle). The labelling is derived algorithmically from the barometer signal and then optionally corrected by hand (§2.6).

2.3.1 Barometer-Based Algorithm

The detector lives in `src/algorithms/segmentation_algorithms/barometer_only/height_segmentation.p` and runs in six stages. Inputs: a time series (t_i, p_i) of timestamps and pressure readings.

1. **Altitude.** Convert pressure to altitude via the international barometric formula,

$$h_i = \frac{T_0}{L}\left[1 - (p_i/p_0)^{RL/(gM)}\right],$$

   implemented in `src/physics/pressure_to_altitude.py`. This removes the weather-dependent DC offset only approximately, but the segmenter depends on *differences* in h, so the residual bias cancels.

2. **Low-pass smoothing.** Apply a centered rolling-mean filter of width `height_lowpass_sec` $\cdot f_s$ samples to suppress high-frequency pressure jitter (door slams, HVAC bursts).

3. **Vertical velocity.** Estimate v_z by forward differencing the smoothed altitude, $v_{z,i} = (h^{\text{lp}}_{i+1} - h^{\text{lp}}_i)/\Delta t_i$, then box-smooth again over `smooth_window_sec`.

4. **Thresholding.** Mark samples where $|v_z| >$ `velocity_threshold` (default ≈ 0.15 m/s) as "moving". Extract contiguous runs of `moving` samples by edge-finding on the binary mask.

5. **Merging.** Walk the runs in order and merge consecutive runs of the *same sign* (both `up` or both `down`) if the gap between them is shorter than `merge_gap_sec`. This collapses spurious velocity dips caused by elevator deceleration mid-ride.

6. **Pruning and padding.** Drop any surviving run whose duration is $\leq$ `min_duration_sec` or whose total altitude change is $\leq$ `min_height_diff_m`. Expand every surviving run by `pad_sec` on both ends so the label comfortably covers the full ride, including the small door-open tail.

The output is a `DataFrame` of rows `(start_ci, end_ci, duration, type, probability_ci)` where `type` $\in$ {`up`, `down`}. The pipeline then pads the gaps between these rows with `outside` intervals so that the GT timeline is contiguous — this is what downstream evaluation code expects.

Why this is good enough as ground truth. A well-calibrated consumer barometer on a phone resolves pressure changes equivalent to roughly 10 cm of altitude at typical sampling rates, which is an order of magnitude below the smallest interesting inter-floor height ($\sim$3 m). The algorithm is deterministic, so repeated runs on the same recording produce identical labels, which in turn makes regression testing tractable.

2.4 The Time-Synchronisation Problem

The reason Section 2.3 alone does not close the loop is a low-level Android timing issue that we spent considerable time getting right.

The root cause. Android's `SensorEvent.timestamp` is the time at which the sensor event was recorded, but it is expressed relative to the *device boot time* (`SystemClock.elapsedRealtimeNanos`), not to wall-clock. The logger writes this timestamp directly into the log file. Consequently, the `timestamp_ms` column in `sensorLog_*.txt` is a *device-local uptime* value, not a UTC time. Two phones that were booted at different moments will assign different timestamps to the *same* physical event, with the offset equal to the difference of their boot times — in our dataset, frequently tens of thousands of seconds, sometimes days.

Why this bites us in practice. Recall (§2.2) that phones without a barometer rely on a secondary device's `PRS` stream for ground truth. To use the secondary device's altitude as ground truth for the primary device, we must answer: *at primary timestamp t_p, what was the altitude?* That requires mapping primary time to secondary time, which requires knowing the boot-time offset between the two devices. Neither device records wall-clock, so the offset must be recovered from the data itself.

First approximation: start-time alignment. The simplest workable estimate is to assume both phones started recording at the same wall-clock moment, so the first sample of each log corresponds to the same real time. Under that assumption, the offset is

$$\Delta = t_0^{(\text{prim})} - t_0^{(\text{sec})},$$

and we shift every secondary timestamp by Δ before using it. This is implemented in `src/data/loader/alignme` and is sufficient when the experimenter hits "start" on both phones in quick succession.

Why start-time alignment is not enough. In practice the user latency between the two taps is one to several seconds and is variable between rides. That multi-second bias shifts the ground truth by enough to invalidate per-sample evaluation (an elevator moves at ~ 1.5 m/s; a 2 s timing error translates to ~ 3 m of altitude error, more than an entire floor). We therefore refine the offset using the **accelerometer signal**, which is recorded by both phones and must be approximately identical for two devices in the same hand.

Refinement by cross-sensor correlation. The routine used in the lab is: (i) compute the magnitude $|a|(t)$ on both phones, (ii) resample both to a common grid after applying the first-pass start-time offset, (iii) run a normalised cross-correlation of the two $|a|$ traces over a bounded lag window (± 10 s was enough in every experiment we observed), and (iv) pick the lag that maximises the correlation as the residual offset to fold back into Δ. If ACC is missing on either device, we fall back to gyroscope magnitude or heuristic event matching on step-count spikes. The diagnostic overlay (Figure 2) is produced automatically for every experiment with a `forBarometer/` companion and is the first thing to inspect when an alignment looks suspicious.

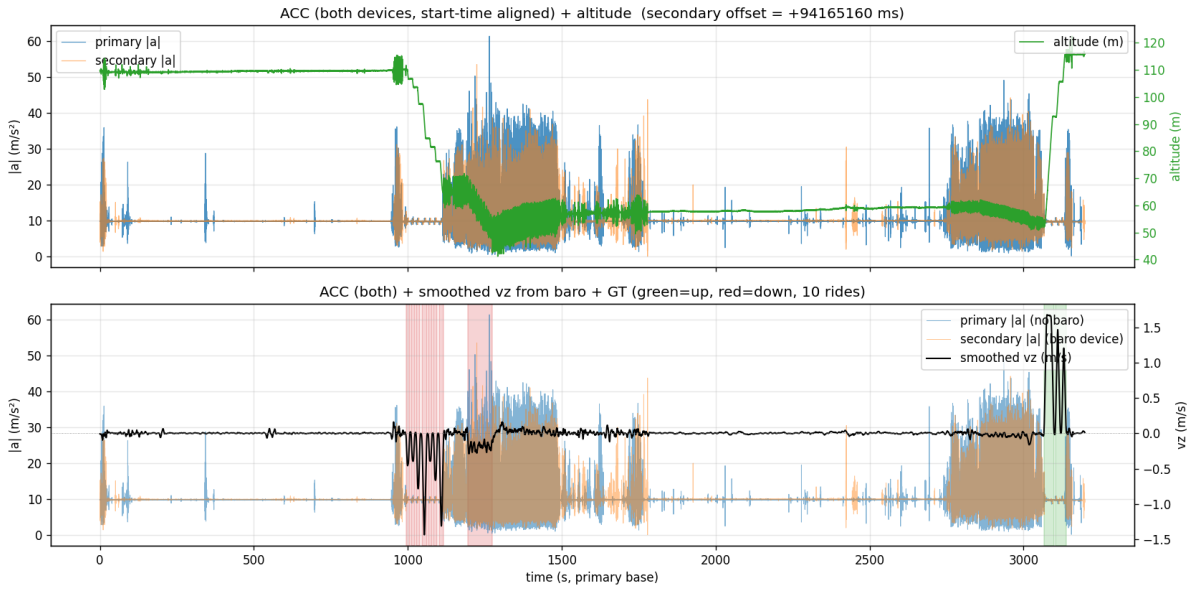


Figure 2: Diagnostic plot produced by `loader/alignment.py::_plot_forBarometer_alignment`. **Top:** primary $|a|$ (blue) and time-shifted secondary $|a|$ (orange) overlaid, with the secondary's altitude on the right y-axis (green). Signature features such as the impulse at door-close should line up after alignment. **Bottom:** the same accelerometer overlay with the barometer-derived smoothed vertical velocity in black and the detected ride intervals shaded (green = up, red = down). The reported offset is the value added to the secondary's timestamps.

Practical hand-offs. The offset is computed once per experiment, logged, and baked into the secondary's `PRS` frame before it is swapped into the primary's sensor dictionary (`_merge_secondary_prs` in `alignment.py`). Everything downstream — the segmenter, the GT derivation, evaluation metrics — therefore sees *a single phone* with a valid barometer, which removes a whole class of timing bugs from the rest of the stack.

2.5 The Loader Pipeline

`src/data/loader/` exposes a narrow public API (`getExperimentData`, `saveExperimentData`, `list_experiments`) that hides all of the above. On first access it parses the raw `sensorLog_*.txt`, merges any `forBarometer/` secondary, runs the detector (§2.3.1), pads the timeline with `outside` intervals, and writes per-sensor CSVs plus `gt.csv` to `structuredData/data/<exp_name>/`. On every subsequent call it reads those CSVs directly. To force a rebuild after code changes, pass `use_cache=False`. The CSV schemas are documented in `src/data/README.md` and frozen in `loader/constants.py`.

2.6 The Ground-Truth Editor

No matter how careful the detector in §2.3.1 is, it will occasionally miss the start of a ride, split a single ride into two, or classify an HVAC-induced pressure ripple as movement. Rather than chase every edge case in the algorithm, we decided early on that the ground truth must be *auditable and editable* by a human. Every `gt.csv` in the dataset has therefore passed through the GT editor at least once.

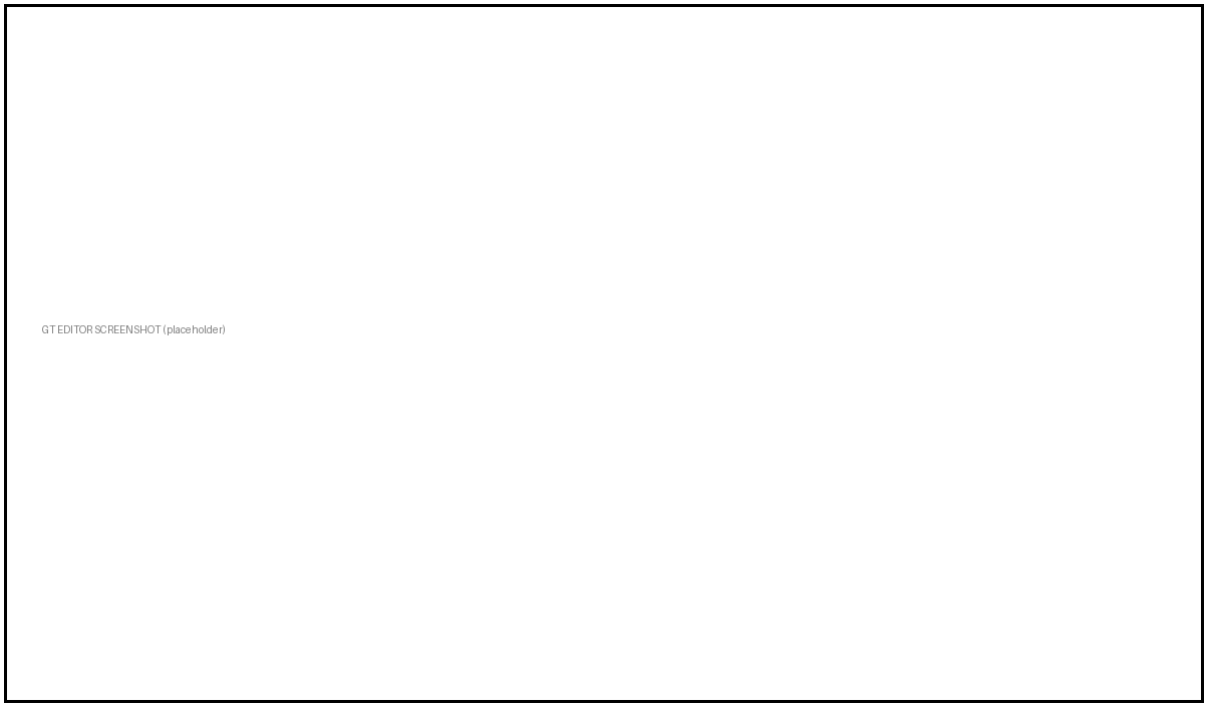


Figure 3: The GT editor. Left pane: stacked matplotlib panels (altitude, vertical velocity, $|a|$, $|\omega|$, $|m|$) with ground-truth intervals drawn as coloured spans (green = up, red = down, grey = outside). Right pane: a `Treeview` of all intervals with the edit form below it.

[TODO: drop the actual screenshot at `docs/latex/figures/gt_editor_screenshot.png`.]

Launching. The editor is a Tkinter application:

```
venv/bin/python -m src.data.gt_editor [exp_folder_name]
```

Called without arguments, it lists all experiments found under `rawData/` in a dropdown and lets the user pick one.

Visualisation. The left pane renders all five standard motion signals on a shared time axis. Ground-truth intervals are overlaid on every panel simultaneously so a user can sanity-check a label against multiple modalities at once (e.g. a true elevator ride must be accompanied by a clear altitude step *and* a flat-ish accelerometer magnitude).

Editing. The interaction model is designed to keep the mouse near the signal rather than in a form. Concretely:

- **Click** anywhere inside a coloured span to select that interval in the right-hand tree.
- **Drag an edge** (within a few pixels of the start or end of a span) to resize it live; the cursor morphs into a horizontal arrow while inside the hit-tolerance zone.
- Type precise millisecond bounds in the **edit form**, change the `type` (`up` / `down` / `outside`) or the free-text `note`, and hit **Apply**.
- + **Add** inserts a new 10-second `outside` interval right after the current selection.
- × **Delete** removes the current interval.

- **Auto-fix** closes gaps between adjacent intervals with `outside` filler so the timeline stays contiguous; it refuses to merge if two intervals actually overlap, to avoid silently destroying user edits.

Navigation. Keyboard shortcuts mirror the matplotlib toolbar: `+` / `=` zoom in, `-` zoom out, `0` fit to full range; `Shift+`← / `Shift+`→ pan by 25% of the visible width; mouse-wheel over the plot zooms around the cursor. All of this is indispensable on long recordings where a single ride is only a few percent of the timeline.

Saving. `Ctrl+S` (or the **Save** button) calls `saveExperimentData`, which rewrites every per-sensor CSV, `gt.csv`, and `metadata.csv` under `structuredData/data/<name>/`, then rebuilds the top-level `metadata.csv` index. The floor-height lookup (`baramoshka.csv`) is intentionally *not* touched on save. The status bar at the bottom tracks unsaved-changes state; closing the window with pending edits prompts before quitting.

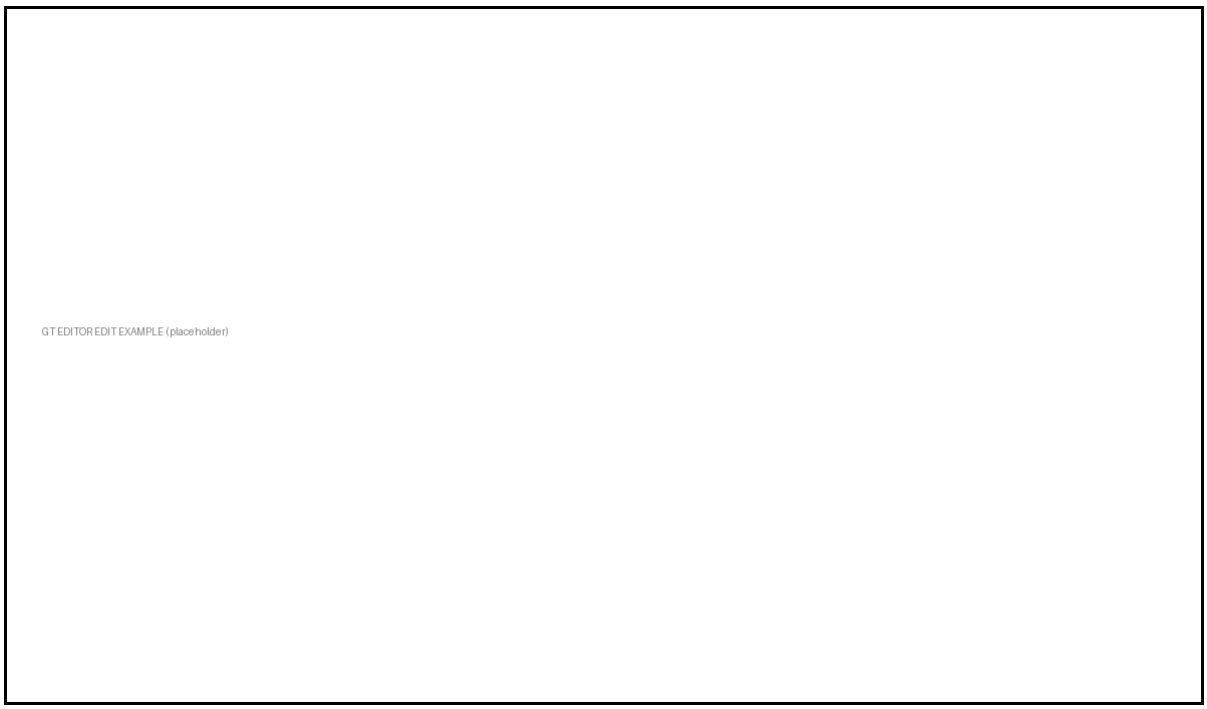


Figure 4: Fixing an auto-detection mistake in the GT editor. Here the barometer algorithm merged two adjacent rides into one because the cabin paused for less than `merge_gap_sec` at an intermediate floor; dragging the right edge of the first span and adding a new `down` interval splits the ride into its two true halves.

[TODO: drop an actual edit-example screenshot at `docs/latex/figures/gt_editor_edit_example`

3 Segmentation & Detection

This section covers how a continuous accelerometer (and, when available, barometer) stream is sliced into elevator-ride segments. We begin with the underlying physics of an elevator ride — a four- parameter motion model that every detector in this work eventually reduces to — and then describe the concrete detectors built on top.

3.1 Background: Elevator Kinematics

Before describing any detector we lay down the physics of an elevator ride, because the same four parameters reappear in every algorithm we use later. Modern passenger elevators are commanded by a *jerk-limited motion profile* so that passengers experience a smooth, bounded change of acceleration (see, e.g., the ride-comfort specifications summarised in ISO 18738 and the manufacturer literature aggregated in `papers/elevator_research_links.txt`). The drive obeys, in order, the constraints

$$|\dddot{h}(t)| \le j_{\max}, \qquad |\ddot{h}(t)| \le a_{\max}, \qquad |\dot{h}(t)| \le v_{\max}, \tag{1}$$

where $h(t)$ is the cabin altitude, $\dot{h} = v$ the vertical velocity, $\ddot{h} = a$ the acceleration measured by the phone (after gravity removal), and $\dddot{h} = j$ the jerk. Typical commercial values are $j_{\max} \approx 1.0\text{–}1.6$ m/s^3, $a_{\max} \approx 0.8\text{–}1.5$ m/s^2, and $v_{\max} \approx 1.0\text{–}2.5$ m/s for low- and mid-rise buildings (CIBSE Guide D, 2020).

The seven-segment S-curve. Saturating each constraint in turn yields the canonical S-curve velocity profile (Figure 5). The ride decomposes into up to seven phases:

$$\underbrace{j = +j_{\max}}_{\text{1: jerk up}} \quad \underbrace{j = 0}_{\text{2: cruise } a_{\max}} \quad \underbrace{j = -j_{\max}}_{\text{3: jerk down}} \quad \underbrace{a = 0}_{\text{4: cruise } v_{\max}} \quad \underbrace{j = -j_{\max}}_{\text{5: jerk down}} \quad \underbrace{j = 0}_{\text{6: cruise } -a_{\max}} \quad \underbrace{j = +j_{\max}}_{\text{7: jerk up}}.$$

Phase 4 collapses to zero length on short hops where the cabin never reaches $v_{\max}$, in which case the profile is "triangular" in acceleration; longer rides are "trapezoidal". The constituent durations follow from (1):

$$T_j = \frac{a_{\max}}{j_{\max}}, \qquad T_a = \max\!\left(0,\ \frac{v_{\max}}{a_{\max}} - T_j\right), \qquad T_v = \max\!\left(0,\ \frac{H}{v_{\max}} - T_j - T_a\right), \tag{2}$$

where H is the floor-to-floor height. The total ride length is $T_{\text{ride}} = 4T_j + 2T_a + T_v$.

What the phone measures. The phone's gravity-projected linear acceleration $a(t)$ traces *exactly* the bracketed acceleration in (1), modulo sensor noise and projection error. Three model facts are worth highlighting because the detector exploits them:

- During the four jerk phases (T_j each), $a(t)$ is linear in time with slope $\pm j_{\max}$. In the velocity domain this integrates to a quadratic — the parabolic "ramp shoulders" the user can see in Figures 5 and 6.
- The signed area under the first acceleration lobe equals the peak velocity reached:

$$v_{\text{peak}} \;=\; \int_0^{2T_j+T_a} a(t)\,dt \;=\; a_{\max}\,(T_j + T_a) \;=\; \min\!\Big(v_{\max},\ \sqrt{a_{\max} H}\Big) \quad \text{for short rides.} \tag{3}$$

  The second (deceleration) lobe has equal magnitude and opposite sign, so $\int_0^{T_{\text{ride}}} a(t)\,dt = 0$. This is the fundamental reason ZUPT works: the cabin starts and ends at rest, so any non-zero terminal velocity in the integrated signal is drift, not motion.
- The temporal gap between the centroids of the two acceleration lobes is the cruise time $T_v + 2T_a + 2T_j \approx H/v_{\max}$ when $v_{\max}$ is reached. **Hence the inter-pulse spacing is a direct, scale-free measurement of the floor-to-floor height H.**

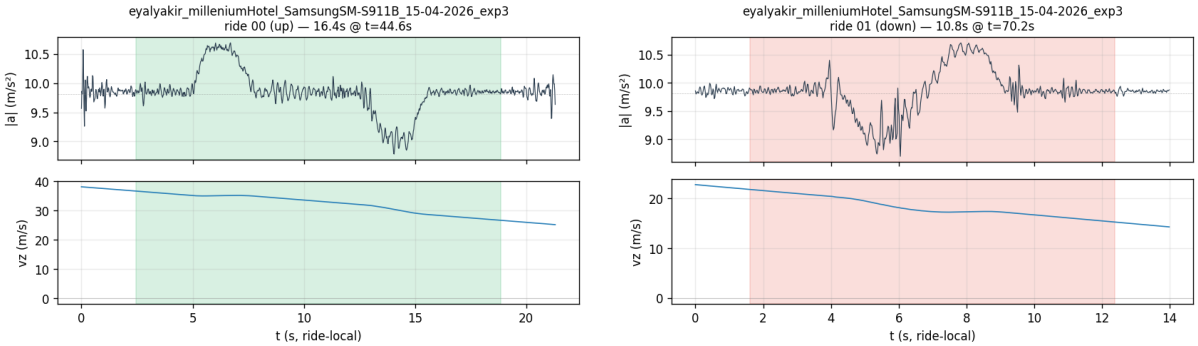
The four-parameter ride model. Combining (1)–(3), every ride is parameterised by the tuple

$$\boxed{\theta_{\text{ride}} \;=\; (j_{\max},\; a_{\max},\; v_{\max},\; H)}\,, \tag{4}$$

with $j_{\max}, a_{\max}, v_{\max}$ being properties of the elevator machine (constant within a building) and H being the only ride- specific quantity. Once θ_{ride} is fixed, the ideal acceleration template is fully determined:

$$a_\theta(t) = \begin{cases} +j_{\max}\, t, & 0 \le t < T_j, \\ +a_{\max}, & T_j \le t < T_j + T_a, \\ +a_{\max} - j_{\max}(t - T_j - T_a), & T_j + T_a \le t < 2T_j + T_a, \\ 0, & 2T_j + T_a \le t < 2T_j + T_a + T_v, \\ \text{(mirror of phases 1–3, with opposite sign)}, & \text{otherwise.} \end{cases} \tag{5}$$

Velocity $v_\theta(t) = \int_0^t a_\theta(\tau)\, d\tau$ and displacement $h_\theta(t) = \int_0^t v_\theta(\tau)\, d\tau$ are then piecewise cubic and quartic respectively. Equation (5) is the synthetic template used by the DTW-based detector (Algorithm 2 in `papers/algorithm2_dtw.md`) and by the matched-filter pulse detector in `template_match/scripts/acc_pulse_detect.py`.



(a) Ride going *up*: positive accel lobe first, negative lobe at landing. The shaded interval is the GT label.

(b) Ride going *down*: lobes are mirrored. The vertical velocity panel makes the trapezoidal $v(t)$ profile clear.

Figure 5: Two single-ride zooms drawn from `eyalyakir_milleniumHotel_SamsungSM-S911B_15-04-2026_exp`. Top of each subfigure: gravity-removed $|a|$. Bottom: the smoothed vertical velocity v_z recovered by integration. The two parabolic lobes in $|a|$ are the jerk-limited acceleration and deceleration phases of (5); the area under each lobe equals v_{peak} via (3), and the gap between lobe centroids measures the cruise time T_v, which is proportional to the inter-floor height H.

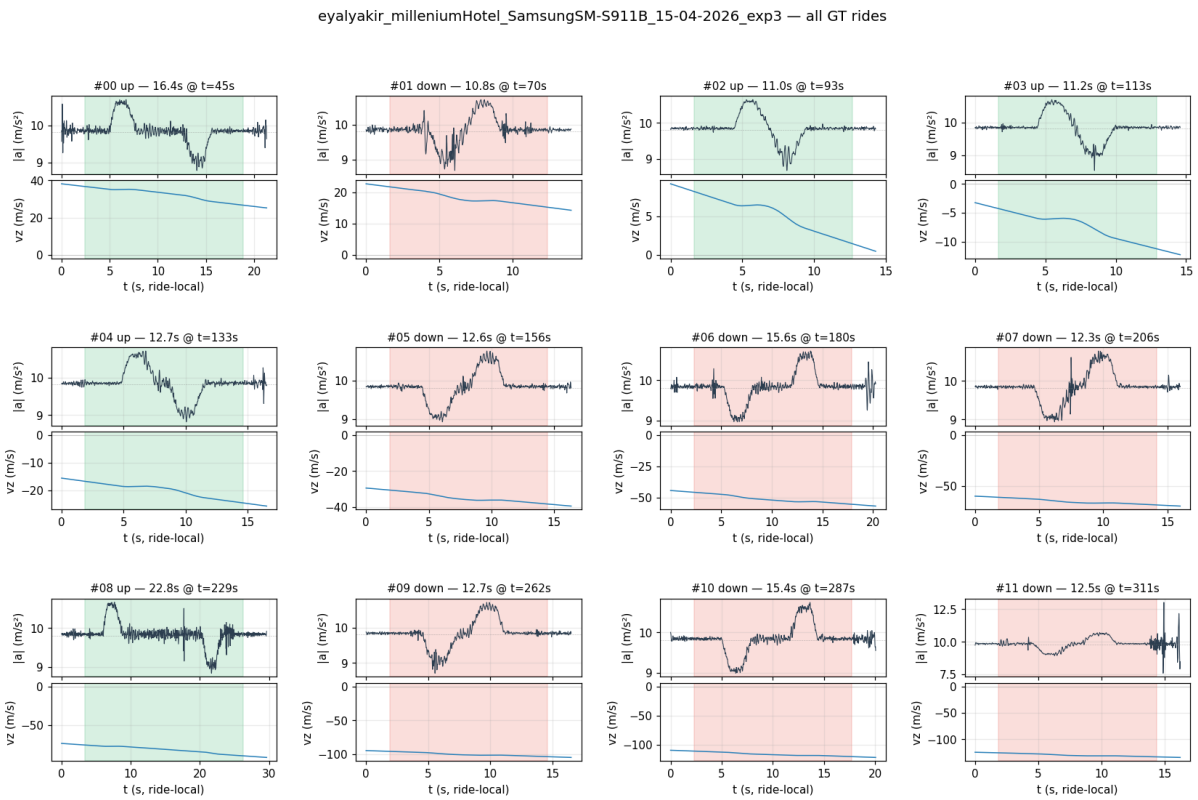


Figure 6: All twelve GT rides from the same experiment, on a common layout. The repeating two-lobe "$+a$ then $-a$" structure predicted by (5) is unmistakable across rides of varying duration. Wider lobe spacing corresponds to larger H (more floors traversed), while the lobe *height* saturates at the building's $a_{\max}$ regardless of H.

Implications for detection. The four-parameter model (4) gives every detector built on top concrete physical knobs:

- State-machine detectors (Algorithm 1 in `papers/algorithm1_state_machine.md`) threshold on $a_{\max}$ to find lobe entries and on a pulse-duration floor of $\sim T_j$.
- DTW-based template matching (Algorithm 2 in `papers/algorithm2_dtw.md`) generates one template per plausible θ_{ride} and warps it.
- Velocity-integral / ZUPT detectors (Algorithm 3 in `papers/algorithm3_integral_zupt.md`) check the integral identity $\int a\,dt = 0$ and the lower bound $\int |v|\,dt \gtrsim H_{\min}$.

The barometer-based GT detector described in §2.3.1 uses none of this — it works in altitude space directly — but the kinematic decomposition above is what makes its output a sensible reference signal for the accelerometer-based algorithms that follow.